

Линейное внимание — конспект лекции

Что предполагается известным. Модуль предполагает знание трансформерного внимания (Q, K, V , softmax по оценкам (scores), каузальная маска, многоголовое внимание); KV-кэша и разделения на пре-филл и декодирование; внимания с группировкой запросов — того, что n_kv_heads и n_heads — разные числа; матричных произведений и их ассоциативности, транспонирования, внешних произведений, ортонормированных базисов и $I - kk^T$ как проектора; среднего и дисперсии скалярного произведения двух случайных единичных векторов; одного шага градиентного спуска на квадратичной ошибке и того, как дифференцировать по матрице; скрытых состояний RNN, гейтов забывания и sigmoid; RMSNorm и каузальной depthwise-свёртки; O-нотации и подсчёта FLOP; и достаточной ментальной модели GPU, чтобы принять, что память на кристалле — маленькая и быстрая, а память устройства — большая и медленная. PyTorch предполагается на семинаре и в домашнем задании — и нигде в этой лекции.

На этой лекции мы познакомимся с большими языковыми моделями, основанными на линейном внимании. В отличие от обычного softmax внимания, линейное внимание позволяет экономить как на вычислениях, так и на операциях с памятью. Чтобы этого достичь, в последние годы был разработан математический аппарат и ряд приёмов по реализации.

Раздел 0. Арифметика KV-кэша

Для начала, давайте вспомним, какие требования к памяти у механизма трансформерного внимания. Для этого рассмотрим формулу внимания для одной головы:

$$O = \text{softmax}(QK^T + M) V$$

где M — каузальная маска. Мы помним, что в работе LLM есть два этапа: префилл и декодирование (prefill и decoding).

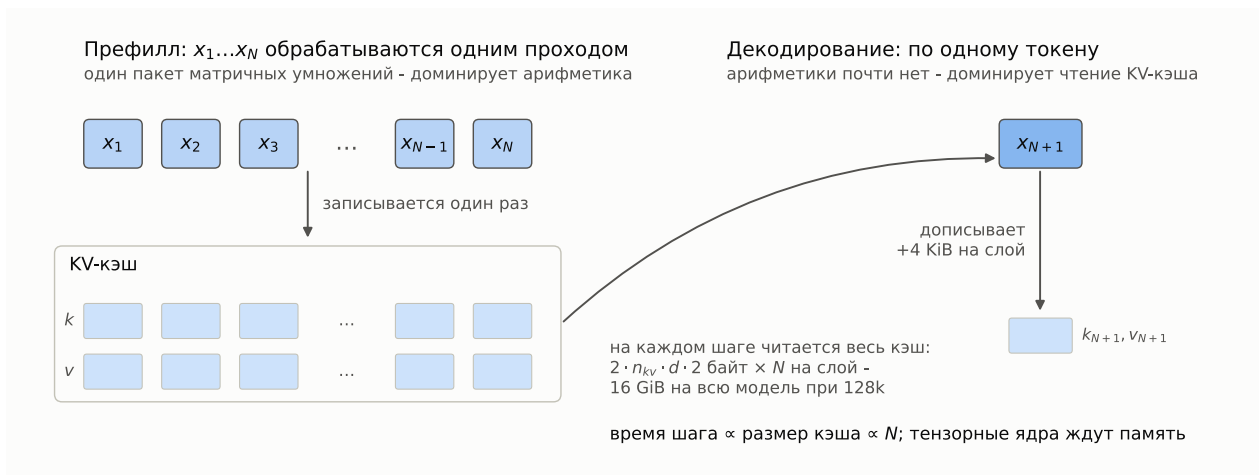


Рис. 1. Префилл и декодирование: что читается из памяти

Первый — префилл: вы отдаёте модели промпт из N токенов, и она обрабатывает их все разом. Это один огромный пакет матричных умножений, железо занято, и в стоимости доминирует арифметика. Второй — декодирование: вы генерируете один токен, потом следующий, потом ещё один, и каждому нужно оглянуться на всё, что было до него. В одном шаге декодирования арифметики, по сути, нет — один токен проходит через сеть, а всё остальное (KV-кэш) уже посчитано. Большая часть работы на шаге декодирования — это чтение KV-кэша из памяти, и именно из-за этого на декодирование уходит всё время. Получается так, что большую часть времени тензорные ядра простаивают, потому что ждут HBM.

Давайте посчитаем, сколько конкретно байтов читает декодирование из памяти вычислителя. Размер KV-кэша одного слоя для N токенов равен

$$2 \cdot n_{kv_heads} \cdot head_dim \cdot dtype_bytes \cdot N$$

Двойка — потому что вы храните и K , и V . Теперь подставим реальную модель. Llama-3-8B: 32 слоя, голов внимания у неё 32, но это модель с группировкой запросов, и только **8** из них — головы ключей и значений. Головы запросов делят их между собой. Так что берём $n_{kv_heads} = 8$, а не $n_{heads} = 32$; $head_dim$ равен 128, а в fp16 это 2 байта.

$$2 \cdot 8 \cdot 128 \cdot 2 = 4096 \text{ байт на токен, на слой}$$

4 KiB на токен, на слой. 32 слоя — значит, 128 KiB на токен для всей модели. При контексте 128k токенов: 512 MiB в одном слое и 16 GiB по всей модели. Это кэш для **одной** последовательности. Не веса — веса примерно столько же ещё раз, и за них вы платили бы в любом случае. Это сверх них, и это на каждого пользователя.

А теперь то, что больно. Каждый шаг декодирования должен прогнать все эти 16 GiB из памяти, чтобы вычислить один токен. На карте с пропускной способностью в несколько сотен гигабайт в секунду это десятки миллисекунд чистого перемещения данных до того, как вообще начнётся хоть какая-то арифметика, и с ходом разговора это линейно ухудшается. Декодирование не упирается в вычисления. Декодирование упирается в память, и то, во что вы упираетесь, растёт вместе с контекстом.

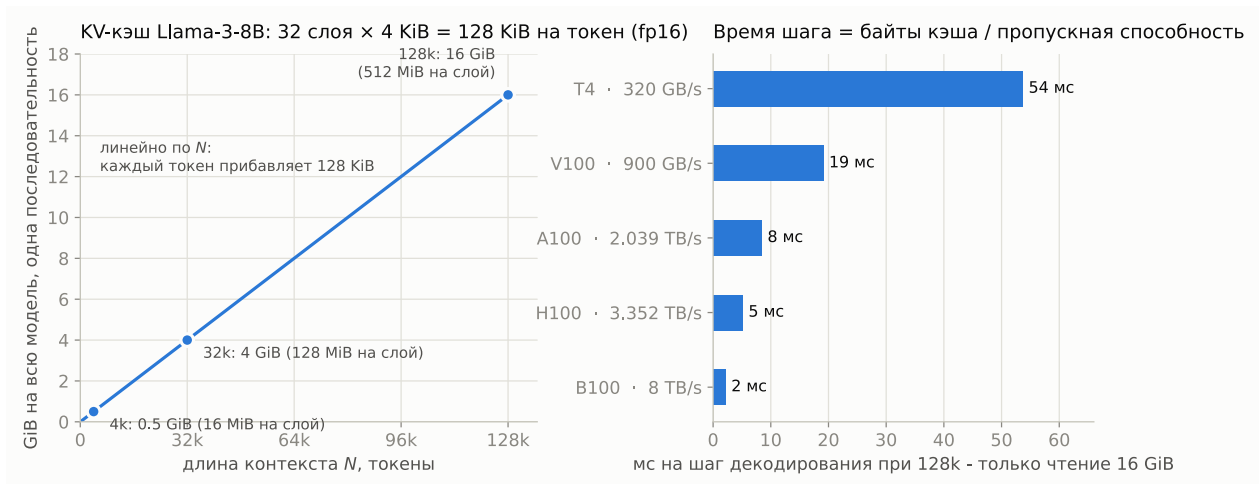


Рис. 2. KV-кэш Llama-3-8B: рост по N и время шага декодирования по картам

Введём две величины, которыми будем пользоваться до конца лекции. Первая — **байты состояния на слой при длине контекста N** : сколько один слой должен держать в памяти после N токенов, чтобы выдать следующий. Для softmax-внимания это размер KV-кэша слоя — 512 MiB при 128k, линейно по N . Вторая — **прирост байтов на токен, на слой**: насколько это число растёт, когда приходит ещё один токен. Для softmax-внимания это 4 KiB, константа.

Итак, вот вопрос на всю оставшуюся лекцию. У softmax-внимания прирост стоимости на токен ненулевой, а состояние растёт без ограничений. **Как можно ограничить рост KV-кэша с увеличением контекста?**

Раздел 1. Линейное внимание и память

Вспомним формулу $O = \text{softmax}(QK^T + M)V$. Softmax просто даёт набор весов, с которыми усредняются строки матрицы V . Выходное значение внимания для токена i , это o_i — i -ая строка матрицы O :

$$o_i = \sum_{j \leq i} v_j \cdot \alpha_{ij}.$$

Вес α_{ij} это софтмакс:

$$\alpha_{ij} = \frac{\exp(q_i^\top k_j)}{\sum_{t \leq i} \exp(q_i^\top k_t)}$$

Тогда

$$o_i = \frac{\sum_{j \leq i} v_j \text{sim}(q_i, k_j)}{\sum_{j \leq i} \text{sim}(q_i, k_j)}$$

где

$$\text{sim}(q, k) = \exp(q^\top k)$$

Это в точности softmax-внимание — мы только вынесли экспоненту наружу и дали ей имя. В таком виде это ядерное сглаживание. Что если мы выберем в качестве sim что-нибудь другое?

Возьмём

$$\text{sim}(q, k) = \phi(q)^\top \phi(k),$$

где $\phi(\cdot)$ — некоторое преобразование признаков. Заметьте: экспонента так *не* факторизуется ни с каким конечным ϕ . Экспонента от скалярного произведения — законное ядро, но для неё отображение ϕ было бы бесконечномерно. Остаются ровно два варианта, и исследователи попробовали оба: приблизить экспоненту конечным случайным отображением признаков и смириться с ошибкой приближения — или отказаться от экспоненты вовсе и выбрать ϕ просто потому, что так удобно.

Подставим факторизованное сходство в числитель и посмотрим, что можно вынести за знак суммы:

$$\begin{aligned} \sum_{j \leq i} \text{sim}(q_i, k_j) v_j &= \sum_{j \leq i} (\phi(q_i)^\top \phi(k_j)) v_j = \sum_{j \leq i} v_j \phi(k_j)^\top \phi(q_i) \\ &= \left(\sum_{j \leq i} v_j \phi(k_j)^\top \right) \phi(q_i) = \left(\sum_{j \leq i} \phi(k_j) v_j^\top \right)^\top \phi(q_i) \end{aligned}$$

В первой строке скаляр $\phi(q_i)^\top \phi(k_j)$ переставлен за вектор v_j — скаляр можно писать с любой стороны. Во второй $\phi(q_i)$ не зависит от j и выходит из-под суммы. Знаменатель преобразуется так же: $\sum_{j \leq i} \phi(q_i)^\top \phi(k_j) = \phi(q_i)^\top \sum_{j \leq i} \phi(k_j)$ — запрос, умноженный на вектор, накопленный по ключам.

В матричной форме, на секунду забыв о каузальности и знаменателе, весь слой — это одно и то же произведение, расставленное по скобкам двумя способами:

$$O = (\phi(Q) \phi(K)^\top) V \quad \text{и} \quad O = \phi(Q) (\phi(K)^\top V)$$

Всю работу здесь делает ассоциативность матричного умножения.

Слева запрос взаимодействует с каждым ключом по отдельности, и потом мы складываем. Справа запрос вышел из-под суммы — потому что он не зависит от j , — и внутри осталась сумма **внешних произведений** ключей на значения. В этих внешних произведениях запроса нет вообще. Их можно вычислить один раз и переиспользовать для каждого запроса.

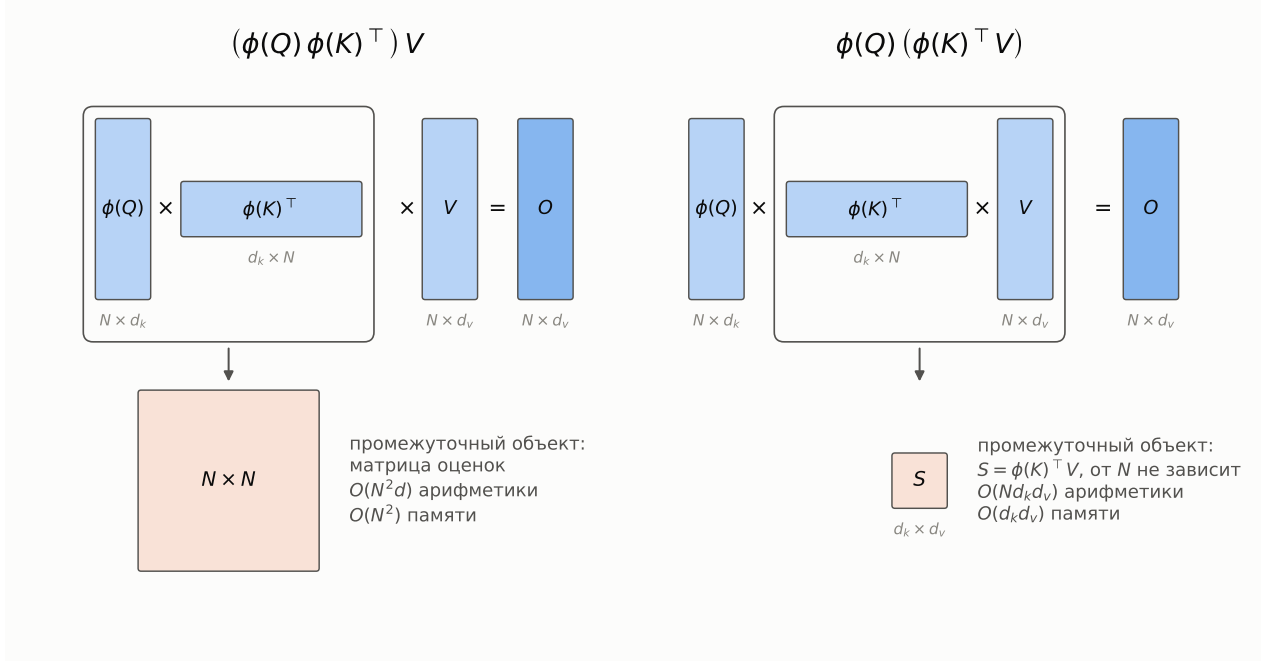


Рис. 3. Две расстановки скобок и размеры промежуточных объектов

Посчитаем стоимость. Левая расстановка скобок образует матрицу оценок $N \times N$: это $O(N^2 d)$ арифметики и $O(N^2)$ памяти — то самое квадратичное внимание, которое вы уже знаете. Правая расстановка вместо этого образует матрицу $d_k \times d_v$: $O(N d_k d_v)$ арифметики и $O(d_k d_v)$ памяти под промежуточный объект.

Линейно по N . По времени — линейно, а для промежуточного объекта — вообще никак: он от N не зависит.

Подставим числа. При $d_k = d_v = 128$ и контексте 128k токенов квадратичный член $N^2 d$ — это около 2×10^{12} умножений-сложений на голову. Линейный член $N d_k d_v$ — около 2×10^9 . Три порядка, при одной длине контекста, в одной голове одного слоя, и отношение равно N/d , так что оно продолжает расти.

Этот промежуточный объект

$$S = \phi(K)^T V, \quad S \in \mathbb{R}^{d_k \times d_v}$$

- матрица фиксированного размера, и это **вся память модели о прошлом**. Всё, что слой когда-либо будет знать о первых N токенах, должно уместиться в d_k на d_v чисел. Softmax-внимание хранит каждый ключ и каждое значение и перечитывает их; эта штука хранит сводку $d_k \times d_v$, а остальное выбрасывает.

Припишем ему два числа. Возьмём линейный слой той ширины, которую реально используют промышленные модели: 32 головы, $d_k = d_v = 128$, 2 байта:

$$32 \cdot 128 \cdot 128 \cdot 2 = 1 \text{ MiB на слой}$$

Байты состояния на слой при длине контекста N : 1 MiB — при любом N . Прирост байтов на токен, на слой: ноль. Размер не меняется, когда приходит токен. Против 4 KiB на токен и 512 MiB при 128k у softmax-внимания это множитель 512 при той одной длине контекста.

Одно замечание, прежде чем идти дальше. Исторически исследователи вкладывали усилия в подбор нужного ϕ . Оригинальный Linear Transformer использовал $\text{elu}(x) + 1$ — выбрано потому, что положительно и дёшево. Performer использовал случайные признаки, чтобы честно приблизить экспоненциальное ядро. Был период, когда каждая статья предлагала новый ϕ .

Вот что произошло: **не победила ни одна ветвь, потому что вопрос был поставлен неверно.** Точно приблизить экспоненту бессмысленно, потому что не экспонента делала softmax-внимание хорошим — его делала хорошим неограниченная память. Лучший ϕ даёт вам лучшее приближение механизма, который вы уже заменили. Современные модели используют тождественное отображение — $\phi(x) = x$ — с L2-нормированными запросами и ключами и слоем нормализации на выходе, а усилия вкладывают в *правило обновления* памяти. Отображение признаков не меняет размер S , а размер S — и есть узкое место.

Раздел 2. Рекуррентная форма и интерференция

Давайте вспомним о каузальности трансформера. Каузальная маска говорит, что позиция i может видеть только позиции $j \leq i$. В перегруппированной форме это значит, что сумма внутри S берётся не по всей последовательности, а по префиксу. Так что в каждой позиции своё S :

$$S_i = \sum_{j \leq i} k_j v_j^\top$$

Переход от $i - 1$ к i добавляет ровно одно слагаемое. Значит,

$$\begin{aligned} S_t &= S_{t-1} + k_t v_t^\top \\ o_t &= S_t^\top q_t \end{aligned}$$

и у нас **рекуррентное соотношение**. Две строки. Это весь слой.

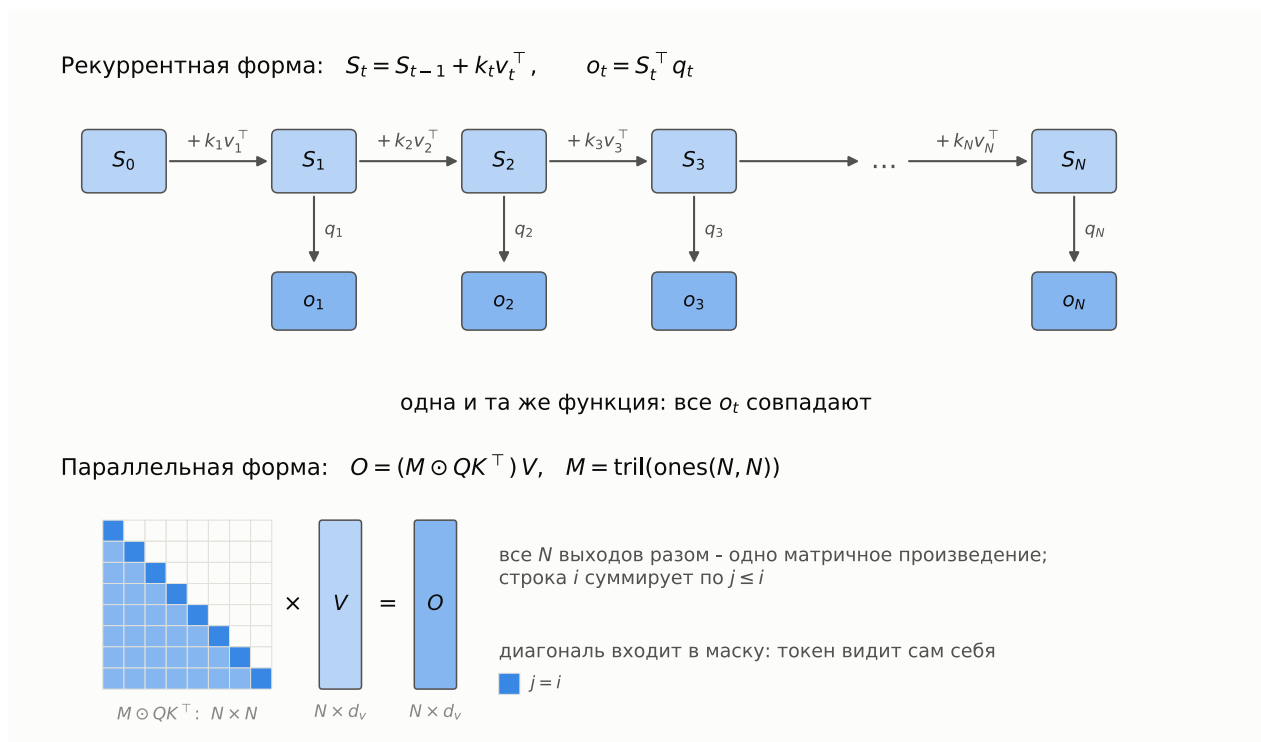


Рис. 4. Рекуррентная и параллельная формы — одна функция

Так что же мы построили на самом деле?

- Это **рекуррентная нейросеть с матричным состоянием**. Обычные RNN поддерживают вектор, эта поддерживает матрицу. Это единственное структурное отличие, и именно поэтому у этой штуки декодирование за $O(1)$: стоимость шага RNN не зависит от того, сколько шагов было до него.
- И это **хеббовская ассоциативная память на внешних произведениях**. Вы записываете пару (k, v) в память, прибавляя $kv^\top$. Вы считываете её обратно, умножая на запрос.

Стоимость декодирования теперь формулируется тривиально. Один шаг — это: сформировать внешнее произведение и прибавить его к S , затем умножить $S^\top$ на q . И то и другое — $d_k d_v$ умножений-сложений.

Вернитесь в блок 0 и сравните. Softmax-декодирование при 128k токенов должно было прогонять 16 GiB на токен, и это число продолжало бы расти, пока длится разговор. Этот слой трогает 1 MiB на слой, 32 MiB по всей модели, причём при любом контексте: 4k токенов, 128k и 1M.

Разложение на сигнал и перекрёстные помехи

Во всей конструкции есть подвох — в считывании, и мы его выведем.

Возьмём состояние после t записей и прочитаем его некоторым запросом q_i :

$$\begin{aligned} S_t^\top q_i &= \left(\sum_{j \leq t} k_j v_j^\top \right)^\top q_i \\ &= \sum_{j \leq t} (k_j v_j^\top)^\top q_i \\ &= \sum_{j \leq t} v_j k_j^\top q_i \\ &= \sum_{j \leq t} (k_j^\top q_i) v_j \end{aligned}$$

Чтение памяти выдаёт вам *каждое* значение, которое вы когда-либо записали, каждое со своим весом — тем, насколько его ключ похож на ваш запрос.

Теперь предположим, вы пытаетесь извлечь значение, сохранённое под ключом k_i , так что запрашиваете чем-то, сонаправленным с k_i . Разобьём сумму:

$$S_t^\top q_i = \underbrace{(k_i^\top q_i)}_{\text{сигнал}} v_i + \underbrace{\sum_{j \neq i} (k_j^\top q_i) v_j}_{\text{помеха}}$$

Первое слагаемое — то, что вы просили. Второе — всё остальное, что вы когда-либо записали, просачивающееся в ответ. Это **перекрёстные помехи** (интерференция), и это не баг реализации — так ведёт себя ассоциативная память фиксированного размера. Вариации такой памяти без помех не существует.

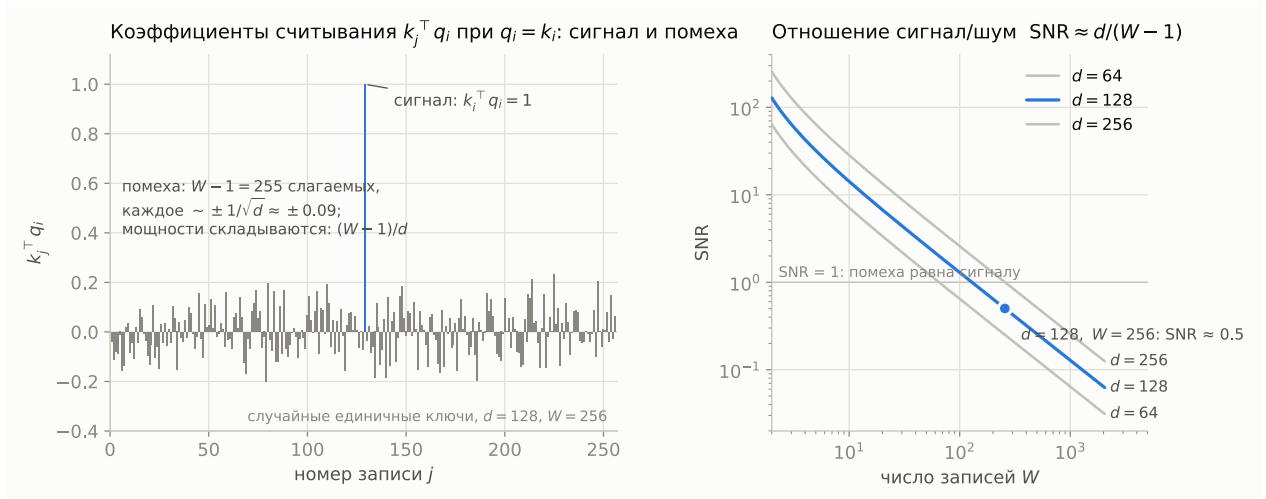


Рис. 5. Сигнал и перекрёстные помехи при считывании; SNR от числа записей

Когда интерференция исчезает? Когда каждое $k_j^\top q_i$ равно нулю при $j \neq i$. Если ключи **ортонормированы** и вы запрашиваете $q_i = k_i$, то каждое перекрёстное слагаемое в точности ноль, и вы получаете обратно v_i , в точности. Идеальное воспроизведение. А теперь посчитайте, сколько ортонормированных векторов существует в $\mathbb{R}^{d_k}$: ровно d_k . Ни одним больше. Заметьте, какая это размерность: d_k ,

ширина *ключа*, а не d_k на d_v . В состоянии $d_k d_v$ чисел, но число вещей, которые оно способно различить, задаётся тем, сколько направлений есть в пространстве ключей.

При промышленных ширинах d_k равно 128. Так что наилучший случай — тот, который не побить никаким выбором ключей, — это 128 ассоциаций на голову. Поразительно маленькое число, если поставить его рядом с окном контекста в 128k токенов.

Реальные ключи не ортонормированы. Это обученные проекции, и в первом приближении они выглядят как случайные единичные векторы. Для случайных единичных векторов в $\mathbb{R}^d$ скалярное произведение двух разных имеет нулевое среднее и дисперсию $1/d$. Так что каждое слагаемое помехи — это маленькое случайное число, умноженное на вектор значения; они некоррелированы, и их мощности складываются. При W записях, стоящих в состоянии, слагаемых помехи $W - 1$, мощность шума $(W - 1)/d$, мощность сигнала — единица, и отношение сигнал/шум вашего извлечения равно

$$\text{SNR} \approx \frac{d}{W - 1}$$

Вот оно. Ёмкость линейна по размерности головы и деградирует по мере того, как вы пишете больше. Это цена конечного состояния в одной строке, и это то, из чего нельзя выбрать оптимизацией.

Подставьте одно число. Возьмите снова $d = 128$ и предположите, что в состоянии стоят 256 записей. Тогда отношение сигнал/шум при извлечении — около 0.5: помеха уже *вдвое* больше сигнала. Не при какой-то экзотической длине контекста — при 256 токенах. Вот масштаб, на котором это становится реальным.

Мы написали W , число **записей**, а не M , число ассоциаций, которые действительно нужно сохранить, и сделали это намеренно. Слой записывает внешнее произведение на *каждом токене без исключения*, несёт этот токен что-то, что вы хотели сохранить, или нет.

Раздел 3. Правило обновления

Всё до сих пор было *линейным* правилом: прибавь внешнее произведение и больше ничего не делай. Это простейшее правило обновления в семействе и, как мы сейчас увидим, ещё и худшее. Весь остальной ландшафт архитектур — это то, что получается, когда это правило пытаются сделать лучше.

План блока: есть **одно уравнение** (линейное обновление), оно сломано, и мы будем его чинить, и у каждой починки есть имя, которое по совпадению оказывается ещё и именем архитектуры.

3.1. Гейт

Начнём с правила, которое у нас есть:

$$S_t = S_{t-1} + k_t v_t^\top$$

и спросим, что происходит спустя долгое время. Каждая запись — навсегда. Каждая запись весит столько же, сколько любая другая. Токен, который вы видели четыре слова назад, и токен, который вы видели 40,000 слов назад, сидят в состоянии с абсолютно равными правами. И по уравнению перекрёстных помех отношение сигнал/шум любого извлечения падает как $d/(W - 1)$ — монотонно, вечно, без дна.

Состояние насыщается. В этом правиле нет ничего, что умело бы забывать, а память, которая не умеет забывать, — это память, которая заполняется и остаётся полной.

Починка очевидна, и это старейшая идея в рекуррентных сетях:

$$S_t = \alpha_t S_{t-1} + k_t v_t^\top$$

Умножьте старое состояние на что-то меньше единицы, прежде чем писать. Запись, сделанная τ шагов назад, ослаблена произведением всех альф с тех пор, так что влияние затухает геометрически, и у состояния появляется эффективный горизонт вместо бесконечного.

В литературе можно выделить три способа реализовать гейт:

- **фиксированный скаляр на голову.** Голова h получает константу γ_h — выбранную, не обучаемую по токенам, вообще не зависящую от данных. Разные головы получают разные константы, чтобы слой покрывал несколько временных масштабов. Это **RetNet**, и его состояние на слой при опубликованной ширине 1.3B — 2 MiB при любом N , с приростом 0 байт на токен.
- **скаляр, зависящий от входа.** α_t вычисляется из текущего токена, так что модель решает, сколько забыть, исходя из того, что только что увидела. Это **Mamba2**, где скаляр выходит из дискретизации пространства состояний как $\exp(-\Delta_t a_h)$ — та же ручка, другая одежда. Её состояние на слой при опубликованном чекпоинте 2.7B — 1.25 MiB при любом N ; прирост 0.
- **вектор, зависящий от входа,** по гейту на каждый канал ключа, так что α_t становится диагональной матрицей и разные строки состояния затухают с разной скоростью. Это **GLA** — gated linear attention, линейное внимание с гейтированием. 1 MiB на слой при любом N ; прирост 0.

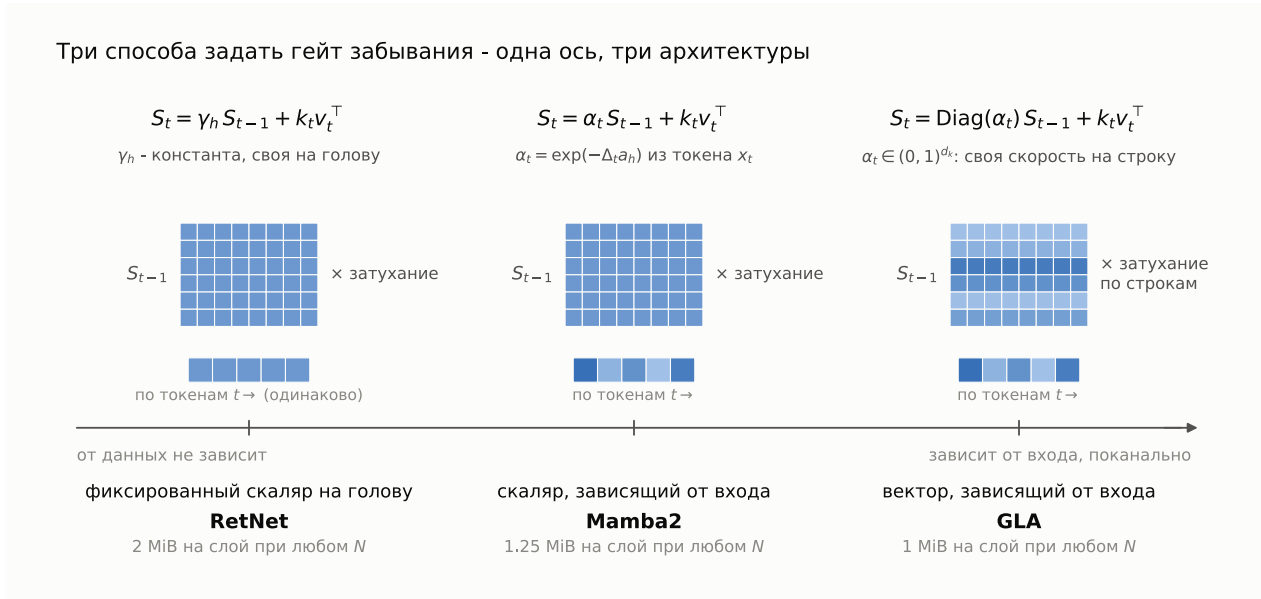


Рис. 6. Три способа задать гейт: RetNet, Mamba2, GLA

Фиксированный скаляр, скаляр от данных, вектор от данных. Вот ось. Архитектуры — ярлыки на точках вдоль неё.

3.2. Дельта-правило

Гейтирование помогает, но посмотрите, что оно может и чего не может. Оно забывает *равномерно*. Оно ослабляет хорошее и плохое с одинаковой скоростью, и у него нет способа сказать «забудь вот эту одну вещь». И есть конкретный провал, которого оно не касается вовсе, и его стоит показать предметно.

Запишите в состояние пару (k, v_1) . Позже в последовательности запишите (k, v_2) — тот же ключ, новое значение. Скажем, в документе было сказано «столица — Бонн», а потом — «столица — Берлин». Что держит состояние?

$$S = k v_1^\top + k v_2^\top = k(v_1 + v_2)^\top$$

и чтение по k даёт $v_1 + v_2$. Не новое значение, а сумму. У правила нет понятия замены, потому что у сложения нет понятия замены. Аддитивные записи ничего не стирают.

Нужно правило, у которого понятие замены есть. Это **дельта-правило**:

$$S_t = (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top, \quad \beta_t \in [0, 1]$$

Имя старое — это правило Уидроу–Хоффа 1960 года. Прочитаем его по частям.

Это перезапись. Положите $\beta_t = 1$ при ключе единичной нормы. Тогда $(I - k k^\top)$ — проектор, убирающий компоненту вдоль k . Так что правило говорит: удали всё, что состояние сейчас ассоциирует с

этим ключом, потом запиши новую ассоциацию. Не прибавить, а заменить. Вернёмся к примеру: после записи (k, v_1) и затем (k, v_2) чтение по k даёт v_2 , а не $v_1 + v_2$. При β_t между 0 и 1 — а в каждой реальной реализации β_t — это сигмоида, так что строго меньше 1, — это частичная замена, интерполяция с обучаемой силой.

Стирающий множитель умножает слева. $(I - \beta_t k_t k_t^\top)$ имеет размер $d_k \times d_k$ и умножает S_{t-1} , размера $d_k \times d_v$, слева. Справа он стоять не может: там нужна матрица $d_v \times d_v$, а она построена из ключей и при $d_k \neq d_v$ даже не согласуется по размерностям; при $d_k = d_v$ выражение согласуется, но стирает в пространстве значений, тогда как стирать нужно в пространстве ключей — там, где живёт k_t .

И это градиентный шаг. У дельта-правила есть второе прочтение. Возьмём функцию потерь

$$\mathcal{L}_t(S) = \frac{1}{2} \|S^\top k_t - v_t\|^2$$

- квадратичную ошибку того, что состояние возвращает по ключу k_t , против того, что оно должно вернуть. Один шаг градиентного спуска по S с шагом β_t — это в точности дельта-правило; вывод — в опциональном подразделе ниже. Так что у слоя есть функция потерь, у него есть шаг обучения, и он делает градиентный шаг на каждом токене. **Слой запускает оптимизатор, а состояние — это модель, которую подгоняют на этапе инференса.** Последовательность — обучающая выборка, правило обновления — оптимизатор, состояние — параметры. Гейтирование пятиминутной давности — это затухание весов (weight decay) на этой внутренней модели.

Архитектура — **DeltaNet**. Размер памяти одного слоя в опубликованном чекпоинте 1.3B — 512 KiB.

Вывод дельта-правила через дифференциал (опционально)

Читается при наличии времени; основная линия лекции от этого подраздела не зависит.

Посмотрите, что за объект $\mathcal{L}_t$. S — линейное отображение из пространства ключей в пространство значений. $S^\top k_t$ — то, что отображение сейчас предсказывает для этого ключа. v_t — то, что оно должно предсказывать. Так что $\mathcal{L}_t$ — квадратичная ошибка предсказания линейной модели на одном обучающем примере, и этот обучающий пример — токен перед нами.

Возьмём производную через дифференциал — так исключаются ошибки в размерностях. Градиент по матрице определяется спариванием Фробениуса

$$d\mathcal{L} = \langle \nabla_S \mathcal{L}, dS \rangle, \quad \langle A, B \rangle = \text{tr}(A^\top B)$$

так что вся задача — записать $d\mathcal{L}$ как что-то, спаренное с dS , и прочитать это что-то. Запишем вектор ошибки

$$e_t = S^\top k_t - v_t, \quad e_t \in \mathbb{R}^{d_v}$$

так что $\mathcal{L}_t = \frac{1}{2} \langle e_t, e_t \rangle$. Двигается только S ; k_t и v_t — константы. Тогда

$$\begin{aligned} d\mathcal{L}_t &= \langle e_t, de_t \rangle = \langle e_t, dS^\top k_t \rangle \\ &= \text{tr}(e_t^\top dS^\top k_t) = \text{tr}(dS^\top k_t e_t^\top) \\ &= \langle dS, k_t e_t^\top \rangle \end{aligned}$$

Средняя строка — единственный ход: скаляр равен своему следу, а след инвариантен относительно циклической перестановки, так что $e_t^\top$ уходит по кругу в конец, и dS остаётся в одиночестве по одну сторону спаривания, а всё остальное собрано по другую. Считываем градиент:

$$\nabla_S \mathcal{L}_t = k_t e_t^\top = k_t (S^\top k_t - v_t)^\top$$

Вектор длины d_k , умноженный на транспонированный вектор длины d_v : матрица $d_k \times d_v$, той же формы, что S .

Теперь сделаем один шаг градиентного спуска с шагом обучения β_t :

$$\begin{aligned}
S_t &= S_{t-1} - \beta_t k_t (S_{t-1}^\top k_t - v_t)^\top \\
&= S_{t-1} - \beta_t k_t k_t^\top S_{t-1} + \beta_t k_t v_t^\top \\
&= (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top
\end{aligned}$$

где в средней строке использовано, что $(S_{t-1}^\top k_t)^\top = k_t^\top S_{t-1}$. Это дельта-правило, записанное выше: стирающий множитель вышел из вывода слева.

3.3 Совмещаем оба

Теперь у нас две независимые починки. Гейтирование забывает глобально; дельта-правило перезаписывает локально. Ни одна не поглощает другую, так что совместим их:

$$S_t = \alpha_t (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top$$

Ослабляем всё состояние, проецируем прочь старую ассоциацию вдоль этого ключа, записываем новую. Это **Gated DeltaNet**. Его опубликованный чекпоинт на 1.3B имеет размер памяти 1.37 MiB при любом N и прирост 0 байт на токен.

И есть уточнение 2026 года, которое стоит назвать, потому что оно показывает, как вся последовательность схлопывается в одно уравнение. **Gated DeltaNet-2** замечает, что дельта-правило привязывает силу стирания и силу записи к одному и тому же скаляру β_t , и спрашивает: почему? Так что оно их развязывает и делает обе поканальными:

$$S_t = (I - k_t (b_t \odot k_t)^\top) \text{Diag}(\alpha_t) S_{t-1} + k_t (w_t \odot v_t)^\top$$

с гейтом стирания b_t из $[0, 1]^{d_k}$ и гейтом записи w_t из $[0, 1]^{d_v}$, где $\odot$ — поэлементное умножение. Стирающий множитель по-прежнему слева.

Вот зачем мы это упомянули. Положите и b_t , и w_t равными β_t , умноженному на вектор из единиц, — и вы получите **KDA**, правило внутри Kimi Linear, с которым мы снова встретимся в блоке 6: 1 MiB на слой при любом N , прирост 0. Схлопните вдобавок α_t в скаляр — и вы вернулись к Gated DeltaNet. Уберите гейт — получите DeltaNet; уберите вместо этого стирающий множитель — получите правило с гейтом; уберите и его — получите линейное правило, с которого мы начинали. **Каждая починка, которую мы сделали в этом блоке, — частный случай этой одной строки.** Сам Gated DeltaNet-2 — это 512 KiB на слой при той ширине, на которой его обучают, прирост 0; правда, заметьте, что статья хранит состояние в fp32, так что в реальном развёртывании получается 1 MiB.

3.4. Нормализация и параметризация

Знаменатель. Почему в реализациях нет деления на $\sum \phi(k)$: при тождественном отображении признаков знаменатель $q^\top \sum_j \phi(k_j)$ — скалярное произведение двух обученных векторов, так что он может быть близок к нулю и может менять знак. Деление на него катастрофически усиливает шум, когда он мал, и переворачивает выход, когда он меняет знак. В теории было бы честно оставить это, поскольку знаменатель был в softmax, но на практике это оказалось лишним. Все модели выбросили этот знаменатель и поставили вместо него RMSNorm на выход, который делает ту работу, ради которой знаменатель и стоял, — контролирует масштаб считывания, — но без смены знака и обнуления. Некоторые добавляют сверху выходной гейт.

Нормировка запросов и ключей. Обычно вектора q и k L2-нормируют перед рекуррентностью. Это не косметика. Именно это даёт $\|k\| = 1$, что в свою очередь даёт $(I - \beta k k^\top)$ собственные значения в $[1 - \beta, 1]$ — сжатие, — так что β ведёт себя как вес интерполяции, а не как нечто, что может промахнуться и взорвать состояние. Подайте в дельта-правило ненормированные ключи — и получите собственные значения ниже минус единицы.

Откуда берутся гейты. β_t — сигмоида от линейной проекции токена. α_t в линии Mamba2 и Gated DeltaNet — это $\exp(-\exp(a)\text{softplus}(\cdot))$, параметризация, единственная задача которой — удерживать затухание в $(0, 1)$, а его логарифм — в приличных пределах. Глубокого смысла тут нет.

И короткая свёртка. Каждая архитектура, которую мы рассмотрели, — DeltaNet, Gated DeltaNet, KDA, Qwen3-Next — ставит короткую каузальную depthwise-свёртку, шириной три или четыре, на q , k и v перед миксером. Выглядит как деталь, но без неё часто ничего не заводится.

Раздел 4. Чанковая форма

Давайте поймём, что мы имеем на данный момент. У нас есть слой с состоянием фиксированного размера и декодированием за $O(1)$. У нас есть пространство правил обновления для него. И у нас есть два способа его вычислить — параллельная форма и рекуррентность, — и это одна и та же функция.

Вот проблема: **обучать нельзя ни одну из них.**

Почему ни одна из двух форм не годится для обучения

Параллельная форма вычисляет

$$O = (M \odot (QK^\top)) V, \quad QK^\top \in \mathbb{R}^{N \times N}$$

Матрица $QK^\top$ стоит $N^2 d_k$ умножений-сложений, умножение её на V — ещё $N^2 d_v$: $O(N^2 d)$ по времени. По памяти — сама матрица оценок, N^2 чисел на голову: $O(N^2)$. Это ровно та стоимость, от которой мы пытались уйти, и это не теоретическое возражение. Подставим: батч на головы $B \cdot H = 8$, $N = 16384$, fp32 — это

$$8 \cdot 16384^2 \cdot 4 \text{ байт} = 8 \text{ GiB}$$

на один тензор оценок, а наивная реализация держит одновременно два — оценки и их маскированную копию: 16 GiB под одни оценки, на практике около 17 GiB со служебными тензорами. Мы построили линейную форму, чтобы избежать квадратичной стоимости, а потом вычислили её квадратично.

Тогда используем рекуррентность. У неё меньше всего FLOP из всего, что мы написали: $4d_k d_v$ операций с плавающей точкой на токен, всего, и никакой зависимости стоимости шага от N . И она катастрофически медленная. Давайте покажем почему.

Посчитаем арифметику и посчитаем перемещение данных для одного рекуррентного шага. Арифметика: $S \leftarrow S + kv^\top$ — это $d_k d_v$ умножений-сложений, и $o = S^\top q$ — ещё $d_k d_v$ умножений-сложений. Умножение-сложение — это 2 операции с плавающей точкой, так что всего $4d_k d_v$ FLOP. Теперь перемещение данных: чтобы сделать этот шаг, нужно прочитать состояние и записать состояние, а состояние — это $d_k d_v$ чисел по 4 байта каждое в fp32. Это $8d_k d_v$ байт. Делим:

$$\text{арифметическая интенсивность} = \frac{4d_k d_v}{8d_k d_v} = 0.5 \text{ FLOP на байт}$$

Половина операции с плавающей точкой на каждый перемещённый байт. И заметьте, что сократилось: размерности. Неважно, насколько широким вы сделаете состояние — интенсивность равна одной второй, всегда. Это не то число, из которого можно выбраться инженерной настройкой; это свойство алгоритма.

Для масштаба: GPU того класса, что мы используем, делает порядка 8 триллионов fp32-операций в секунду и перемещает порядка 300 гигабайт в секунду. Отношение — арифметическая интенсивность, при которой машина сбалансирована, — около 25 FLOP на байт. Мы в 50 раз ниже. Это значит, что рекуррентная форма может использовать в самом лучшем случае около 2% арифметики, на которую способна карта, как бы хорошо она ни была написана. Тензорные ядра простаивают. И вдобавок у неё N последовательных шагов, которые нечем перекрыть: шаг $t + 1$ не может начаться, пока не закончился шаг t . Ограничение не FLOP, а перемещение данных.

Конструкция

Сложим два наблюдения. У рекуррентности память $O(d_k d_v)$, но интенсивность 0.5 и N последовательных шагов — железо простаивает. У параллельной формы вся работа собрана в плотные матричные умножения с высокой интенсивностью, но память $O(N^2)$ — не помещается. Значит, нужна форма, в которой арифметика собрана в матричные умножения, как у параллельной, а память не зависит от

N , как у рекуррентной. Идея следующая: режем последовательность на чанки длины L . Внутри чанка делаем квадратичную вещь — это всего лишь $L \times L$, а L мало. Между чанками делаем рекуррентную вещь — шагов всего N/L .

Опишем формально. Зафиксируем чанк, и пусть S — память после всех токенов до чанка. Пусть Q, K — запросы и ключи чанка, каждый $L \times d_k$, а V — его значения, $L \times d_v$. Возьмём токен i внутри чанка. Его состояние — это S плюс всё, что записано внутри чанка вплоть до i включительно, так что его выход равен

$$o_i = S^\top q_i + \sum_{j \leq i, j \in \text{чанк}} (k_j^\top q_i) v_j$$

Первое слагаемое от j не зависит вовсе — каждый токен в чанке читает одно и то же входящее состояние, только своим запросом. Сложите их стопкой по чанку — и это одно матричное произведение, QS . Второе слагаемое — квадратичная форма, ограниченная чанком и маскированная. Сложите стопкой и это — получите:

$$O = QS + (M \odot (QK^\top))V$$

$$S_{\text{out}} = S + K^\top V$$

где $M = \text{tril}(\text{ones}(L, L))$, а $\odot$ — поэлементное умножение.

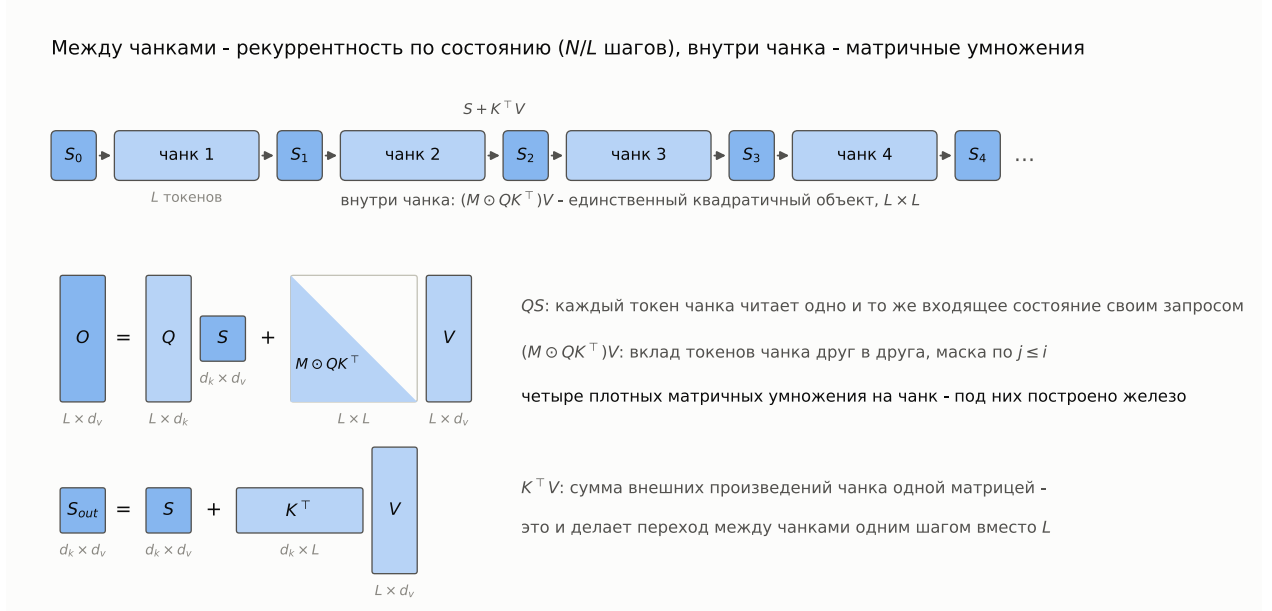


Рис. 7. Чанковая форма: рекуррентность между чанками, матричные умножения внутри

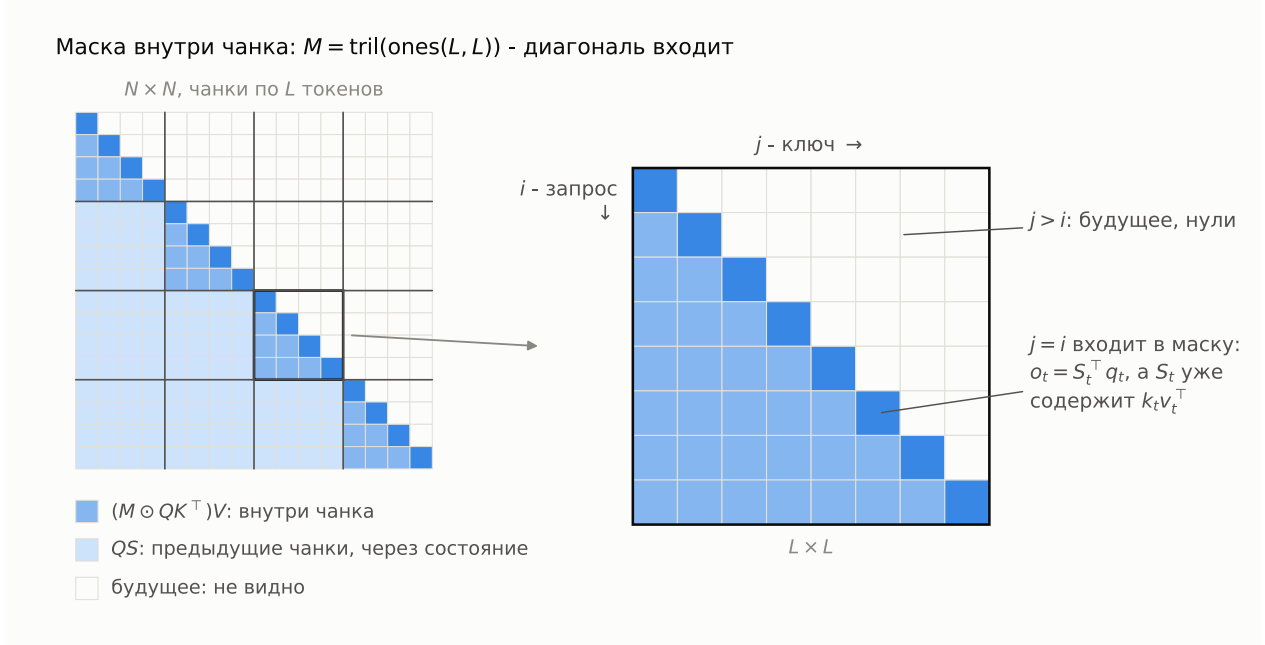


Рис. 8. Маска внутри чанка: диагональ входит; место чанка в каузальной матрице

$QK^\top$, $(M \odot (QK^\top))V$, QS , $K^\top V$ — четыре плотных матричных умножения на чанк, все той формы, под которую построено железо. Рекуррентность не исчезла, а *огрубилась*: с N шагов до N/L шагов, а всё между шагами превратилось в GEMM.

Память следует немедленно. Единственный квадратичный объект во всей этой паре уравнений — $QK^\top$, и он $L \times L$, а не $N \times N$. Так что тензор оценок, из-за которого параллельная форма падала с OOM на 17 GiB, теперь L^2 на чанк: при чанке 128 против последовательности 16k это в 16,000 раз меньше. Мы получили форму параллельной формы и след рекуррентности одновременно.

А теперь снова посчитаем арифметическую интенсивность, для шага по чанку, так же, как считали для рекуррентности. На чанк, при $d_k = d_v = d$: два произведения $L \times L$ стоят $4L^2d$ FLOP, два произведения с состоянием — $4Ld^2$, итого $4Ld(L + d)$. Перемещение данных, если состояние остаётся резидентным, — это Q , K и V на вход и O на выход: $4Ld$ чисел, $16Ld$ байт. Делим:

$$\text{арифметическая интенсивность шага по чанку} = \frac{L + d}{4} \text{ FLOP на байт}$$

Посмотрите, что это говорит. В отличие от плоских 0.5 у рекуррентности, эта величина *растёт* — и с шириной головы, и с длиной чанка.

U-кривая

Посчитаем стоимость. На чанк мы делаем $O(L^2d)$ на внутрочанковую часть и $O(Ld_kd_v)$ на части с состоянием. Чанков N/L , так что в целом

$$O(NLd + Nd_kd_v)$$

Первое слагаемое растёт с L : чанки больше — квадратичной работы больше, и при $L = N$ вы вернулись к полностью квадратичной форме. Второе слагаемое от L не зависит вовсе. Но *число последовательных шагов* — это N/L , которое с ростом L сокращается, и при $L = 1$ вы вернулись к рекуррентности — буквально: тот же путь в коде с одним изменённым аргументом.

Итак, две силы тянут в противоположные стороны, и где-то между ними есть минимум. Это U-кривая, и вы измерите её на семинаре.

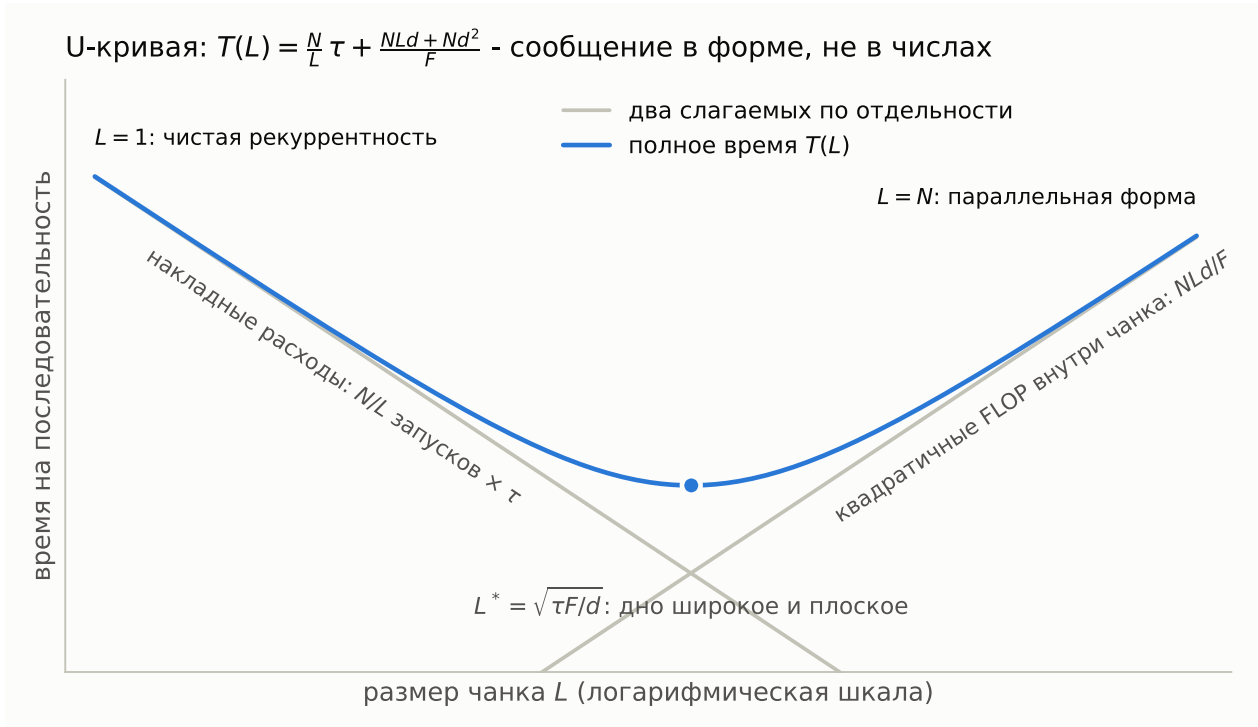


Рис. 9. U-кривая времени от размера чанка

Следует быть осторожным в том, что предсказываем, потому что здесь настоящая ловушка. Смоделируем полное время как N/L чанков, каждый с фиксированными накладными расходами τ , плюс FLOP, делённые на достигнутую скорость F ; FLOP берём из подсчёта выше, с точностью до константы:

$$T(L) = \frac{N}{L} \tau + \frac{NLd + Nd^2}{F}$$

Первое слагаемое убывает по L , второе растёт. Приравняем производную нулю:

$$\frac{dT}{dL} = -\frac{N\tau}{L^2} + \frac{Nd}{F} = 0 \Rightarrow L^* = \sqrt{\frac{\tau F}{d}}$$

Оптимум растёт с накладными расходами на чанк и с достигнутой скоростью и не зависит от N .

А что такое τ ? В реализации на PyTorch τ — это итерация цикла на Python и горстка запусков ядер — скажем, несколько микросекунд. По сравнению с любой арифметикой в чанке это огромно, так что оно сдвигает измеренный оптимум *сильно выше* того, куда его поставил бы чистый FLOP-аргумент, и оно движется вместе с CPU хоста, потому что итерация на Python — это стоимость на CPU. В слитном (fused) ядре один запуск на всю последовательность, и τ почти исчезает, что сдвигает оптимум совсем в другое место.

Итак: **предсказать можно диапазон, в котором лежит оптимум, и причину, по которой он там оказывается, но не само число.** На железе, на котором это строилось, дно U широкое и плоское на размерах чанка примерно от 128 до 256, а $L = 1$ — чистая рекуррентность — стоит примерно в 100 раз больше оптимума. Если измерения дают что-то другое, то первое, что надо проверить, — не код, а протокол замера времени: когда эту кривую измеряли для семинара, положение минимума сдвигалось в зависимости от того, дали ли GPU время выйти на стабильную частоту. Широкий пологий минимум очень легко переместить плохим бенчмаркингом.

С DeltaNet сложнее

Всё, что мы только что сделали, опиралось на одно свойство: обновление состояния в линейном правиле — это **сумма**, так что вклад собственных токенов чанка схлопывается в одну матрицу $K^T V$, а входящее состояние проходит нетронутым.

Теперь попробуйте то же с дельта-правилом. Провести состояние через чанк — значит перемножить матрицы перехода

$$(I - \beta_L k_L k_L^T) \cdots (I - \beta_2 k_2 k_2^T) (I - \beta_1 k_1 k_1^T)$$

а произведения различных единичных матриц с возмущением ранга один **не** схлопываются. Нет одной матрицы, которую можно выписать на глаз. Наивный чанковый вывод попросту не работает.

Это было решено, и это настоящий вклад: Yang, Wang, Zhang, Shen и Kim, *Parallelizing Linear Transformers with the Delta Rule over Sequence Length*, arXiv 2406.06484. Результат не так тривиален, чтобы легко вывести его в рамках данной лекции. Всё сводится к решению СЛАУ с треугольной матрицей.

Упомянуть его стоит ради морали, и мораль — это смысл этого блока. **Чанкование — исследовательский вклад на каждую архитектуру, а не что-то бесплатное.** Каждый раз, когда кто-то предлагает новое правило обновления, кто-то должен вывести его чанковый алгоритм — иначе правило нельзя обучать на больших масштабах, и неважно, насколько оно хорошее. Вот почему эта область исследований движется с той скоростью, с которой движется.

Итак: один объект, три взгляда. Параллельный, рекуррентный, чанковый. Одна функция, три разных профиля арифметической интенсивности.

Раздел 5. Реальность железа

Последние 10 минут ограничением всякий раз оказывалось перемещение данных. Сделаем это конкретным: вторая половина семинара — это GPU-ядро, и входить в неё стоит, зная, что это ядро на самом деле оптимизирует.

Иерархия

У GPU есть иерархия памяти, и числа в ней разнятся на порядки. На карте из Google Colab или Kaggle (NVIDIA T4) 16 GB памяти устройства со скоростью примерно 320 GB/s, 4 MB L2, общего для всех, и затем, на каждый потоковый мультипроцессор, 64 KB разделяемой памяти (shared memory) и регистровый файл, и оба работают с пропускной способностью примерно в 100 раз выше, чем у памяти устройства. Таких мультипроцессоров 40. Карта, на которой разрабатывался этот материал, — Ampere, с 100 KB разделяемой памяти на мультипроцессор вместо 64 KB; на семинаре, где состояние придётся уместить в эту память, разница окажется существенной.

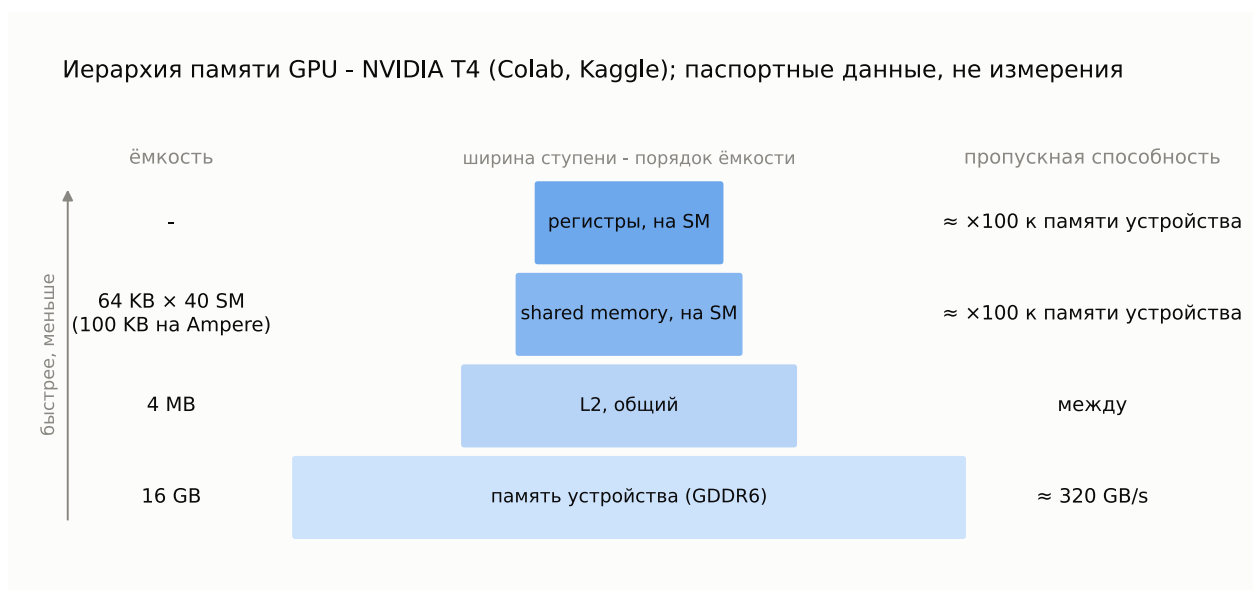


Рис. 10. Иерархия памяти GPU: ёмкость и пропускная способность

Важны два отношения. SRAM примерно на два порядка быстрее памяти устройства и примерно на пять-шесть порядков меньше. Это напряжение — весь предмет инженерии GPU-ядер.

Roofline-модель

Теперь сложим их вместе. Обозначим π — пиковую производительность карты (FLOP/s), β — пропускную способность памяти (байт/s), I — арифметическую интенсивность ядра (FLOP на байт). Скорость, с которой ядро вообще может работать, равна

$$P(I) = \min(\pi, \beta \cdot I)$$

Нарисуйте это как функцию I — и получите линию, поднимающуюся под 45 градусов в логарифмических осях, а затем выходящую на плато на пике машины. Стык двух участков — **точка излома (ridge point)** — отношение пика к пропускной способности. Для этой карты:

$$I^* = \frac{\pi}{\beta} \approx \frac{8 \cdot 10^{12}}{320 \cdot 10^9} = 25 \text{ FLOP на байт}$$

Всё левее точки излома, $I < I^*$, упирается в память: вы ждёте данные, и дополнительная арифметика бесплатна. Всё правее, $I > I^*$, упирается в вычисления: вы ждёте арифметические блоки, и дополнительное перемещение данных бесплатно.

Поставим на неё две формы линейного внимания. Рекуррентная форма — $I = 0.5$, в 50 раз левее точки излома, в области, где арифметика машины вам по сути недоступна, что бы вы ни делали. Шаг по чанку, по подсчёту из блока 4, — $I = (L + d)/4$, что при $L = d = 64$, ширинах семинара, равно 32: прямо в углу. Это иллюстрация основного тезиса блока 4, и вот почему форма, у которой FLOP больше, выигрывает по секундомеру.

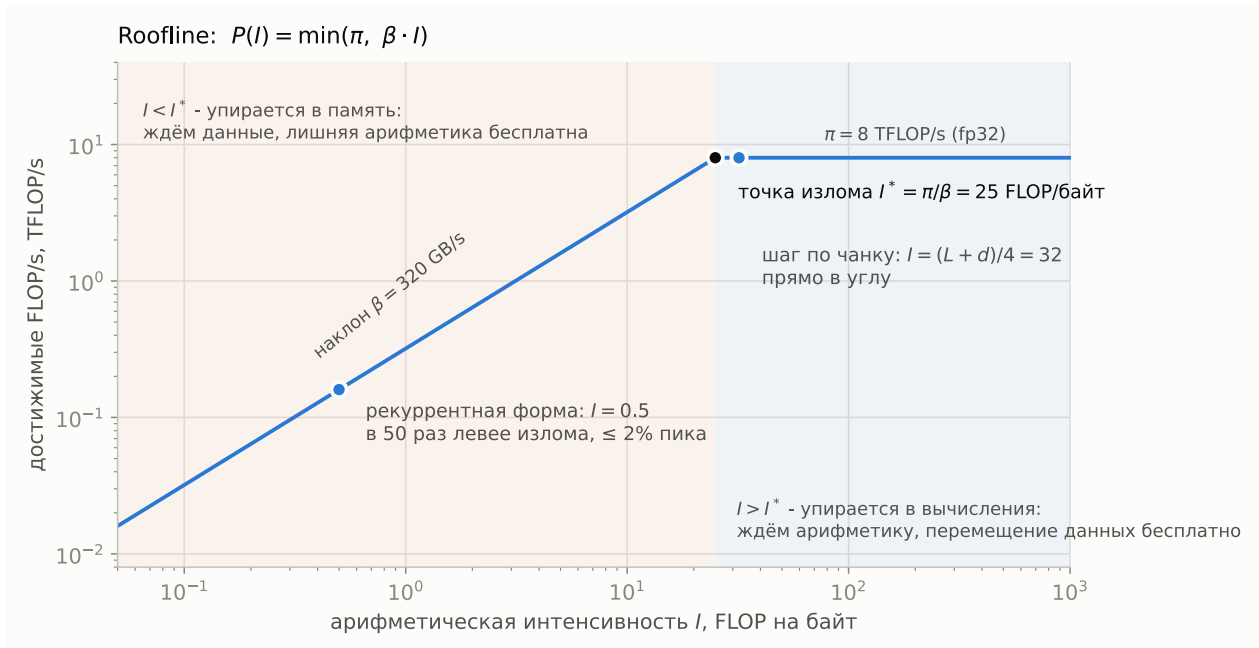


Рис. 11. Roofline-модель: точка излома и две формы линейного внимания

I/O

Вот часть, которую легко упустить. Чанковая форма в наивной реализации на PyTorch делает правильные FLOP, но выполняет каждое матричное умножение как отдельное ядро, так что Q , K , V , блок оценок и состояние между каждой операцией совершают путешествие в память устройства и обратно. Арифметика правильная, а перемещение данных — нет.

Слитное (fused) ядро меняет это и больше ничего. Оно загружает Q , K и V чанка один раз, держит состояние в регистрах **всю последовательность**, вычисляет все четыре произведения, ни разу не записывая промежуточный результат наружу, и сохраняет выход один раз. В идеальном случае память устройства видит каждый элемент Q , K и V ровно один раз, каждый элемент O ровно один раз, а состояние ровно дважды — один раз на входе в начале, один раз на выходе в конце.

Это та же идея, что в FlashAttention, и паттерн стоит назвать, потому что он повторяется повсюду: алгоритм не изменился, изменилось перемещение данных. Подробнее об этом на семинаре.

Раздел 6. Работает ли это?

Пора честно сказать, насколько хорошо всё это работает, потому что ответ — не просто «да».

На общем языковом моделировании линейные архитектуры близки. Уравняйте число параметров и бюджет токенов — и разрыв по перплексии с трансформером достаточно мал, чтобы заголовочные числа их не различали. Если бы вас волновала только перплексия, задача была бы решена, и мы бы все уже перешли.

Разрыв открывается на **ассоциативной памяти**. Всё, что требует воспроизвести конкретный факт из конкретного места: поиск иголки в стоге сена, многошаговые обращения по длинному документу, обучение в контексте на длинном списке примеров, код со ссылками на тысячи токенов назад. На этом линейные модели деградируют, и деградируют характерным образом — не мягко расплываясь, а уверенно извлекая не то.

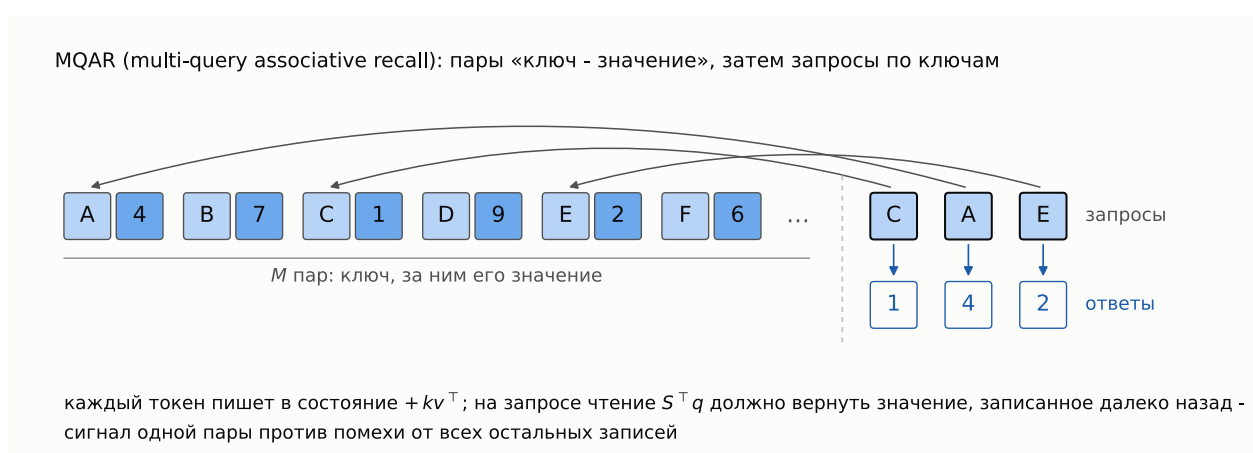


Рис. 12. Последовательность MQAR: пары «ключ — значение», затем запросы

И мы уже знаем почему, потому что вывели это в блоке 2. Перечитайте уравнение перекрёстных помех: извлечение — это слагаемое сигнала плюс слагаемое помехи, и помеха растёт с числом записей, стоящих в состоянии. Затолкайте достаточно ассоциаций в память фиксированного размера — и помеха затопит сигнал.

Оценим степень сжатия. Наш референсный линейный слой держит 1 MiB — 524,288 чисел fp16. Softmax-внимание при 128k токенов держит 512 MiB, на слой. Так что линейный слой просят представить в одной матрице то, что трансформер хранит в 512 раз большем объёме. Процедуры, которая делает это без потерь, не существует. **Это не задача оптимизации.** Больше данных это не починит, более долгое обучение не починит, лучшая инициализация не починит. Это теоретико-информационная граница, и эта граница — ровно то число, которое мы всё утро записываем в байтах.

Если состояние слишком мало для точного воспроизведения, а увеличивать его стоит квадратично, инженерный ответ — перестать пытаться заставить один слой делать обе работы. Используйте в основном линейные слои и вставьте среди них небольшое число слоёв полного внимания, которые возьмут на себя точное воспроизведение.

Qwen3-Next: 48 слоёв, полное внимание в каждом четвёртом, то есть 36 линейных и 12 полных — ровно 3:1. **Kimi Linear:** статья задаёт равномерное 3:1, а поставляемый чекпоинт на 48B — это 20 линейных слоёв на 7 полных из 27 слоёв, что почти, но не совсем 3:1.

Теперь проделаем арифметику для двух введённых величин — байтов состояния на слой и прироста на токен.

Линейные слои Qwen3-Next: 32 головы, $d_k = d_v = 128$, 2 байта — 1 MiB на слой, при любом N , прирост 0. Её слои полного внимания — с группировкой запросов, 2 KV-головы шириной 256, так что $2 \cdot 2 \cdot 256 \cdot 2 = 2$ KiB на токен на слой. Прирост по всей модели: 12 слоёв на 2 KiB = **24 KiB на токен**. Будь каждый слой полным вниманием, это было бы 48 на 2 KiB, то есть 96 KiB. А 24 на 96 — это **ровно 25%**.

Вот что означает «сокращение KV-кэша на 75%». Это утверждение о числе-приросте и ни о чём больше. Вклад 36 линейных слоёв — 36 MiB по всей модели — этим утверждением не затрагивается и с N не двигается вовсе.

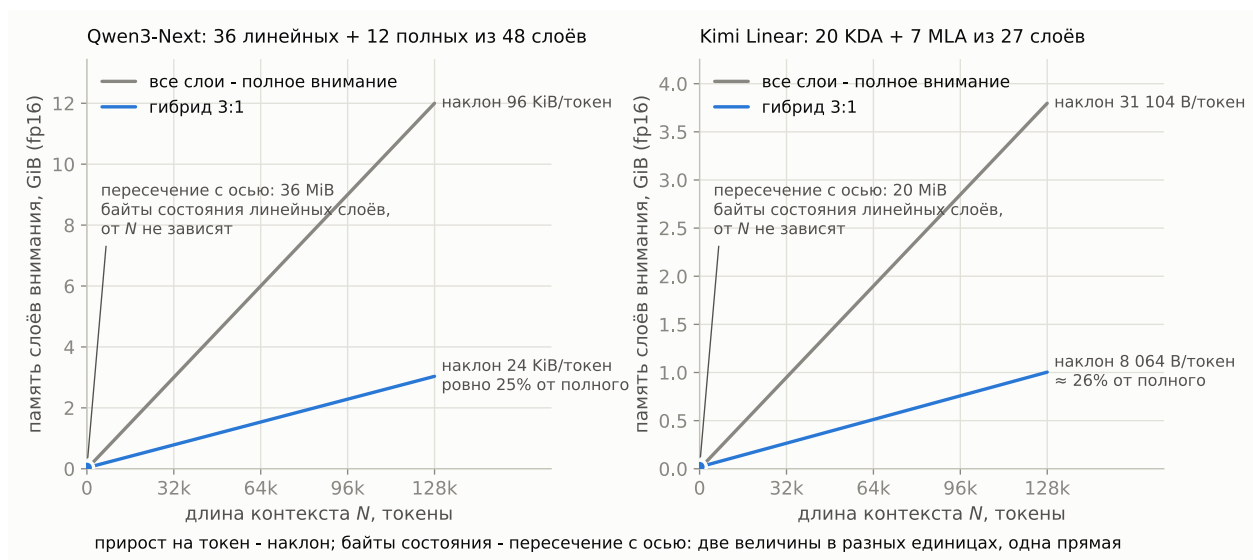


Рис. 13. Память слоёв внимания гибридов от N : наклон и пересечение с осью

Kimi Linear приводит тот же аргумент, и стоит отметить одну вещь, чтобы вы не применили нашу формулу не туда. Её слои полного внимания — **MLA**, которое кэширует один сжатый латентный вектор на токен в слое — шириной 576 чисел, общий для всех 32 голов, так что при 2 байтах на число это 1,152 байта на токен на слой, и с числом голов оно не масштабируется. Наша формула $2 * n_{kv_heads} * head_dim$ его не описывает. 7 таких слоёв дают 8,064 байта на токен против 31,104, будь все 27 слоёв MLA, — сокращение чуть больше 74%, и вот почему статья говорит «до 75%», а не «75%».

Раздел 7. Итог

Выбирайте линейное внимание, когда контекст длинный и платите вы за пропускную способность декодирования; когда развёртывание упирается в память и карту заполняет KV-кэш, а не веса; когда вы обслуживаете много последовательностей одновременно, потому что кэш — на последовательность, и выигрыш тоже; и когда вы можете гибридизировать, а это почти всегда возможно.

Не выбирайте его, когда задача требует точного воспроизведения — поиск по длинным документам, код с далёкими ссылками, транскрипты агентских сессий, полные конкретных фактов, которые должны вернуться дословно; когда контекст короткий, потому что ниже нескольких тысяч токенов кэш мал, и softmax-декодирование на самом деле *дешевле* — вы увидите это пересечение на семинаре, а наш собственный шаг декодирования платит фиксированные несколько сотен микросекунд накладных расходов фреймворка, которые softmax-проход на коротком контексте попросту обгоняет; когда бюджет предобучения мал и вам нужен рецепт без сюрпризов; и когда вам нужно обучать правило, для которого никто не опубликовал чанковый алгоритм, потому что блок 4 — это то, во что это обходится.

2026 год в свою очередь показал: ответ — не «линейное вместо softmax». Ответ — **«в основном линейное, а softmax там, где живёт точное воспроизведение»**.

Открытые проблемы, коротко. Ёмкость на байт состояния — самая живая из них: декаплинг стирания и записи в Gated DeltaNet-2 — шаг ровно в этом направлении, и будут ещё. Чанковые алгоритмы для новых правил: каждому новому правилу обновления нужен свой, каждый — отдельная статья, и

это ограничитель скорости для всего поля. Правильное соотношение и правильное *размещение* слоёв полного внимания не устоялось: собственная абляция Kimi говорит «три к одному», а потом поставляемая модель тихо добавляет ещё один сверху, что говорит вам, что абляция не была последним словом. И даёт ли трактовка через обучение на этапе инференса (test-time training) — состояние как модель, которую подгоняют онлайн, — что-то сверх приятной интерпретации, по-настоящему открыто.

Короткий список литературы, всё в раздатке с полными ссылками: *Transformers are RNNs* — оригинальная конструкция; статья GLA — гейтирование, сделанное как следует; статья Mamba2 о двойственности пространства состояний — отображение между двумя литературами; статья о чанковом DeltaNet — то, что блок 4 лишь упомянул; Gated DeltaNet и Gated DeltaNet-2 — совмещение и его уточнение; Kimi Linear — промышленный гибрид; и обзор 2025 года по эффективному вниманию, если хотите всю таксономию. Репозиторий `flash-linear-attention` — то место, где ядра на самом деле живут.